Миграции и схема

Инструмент

Alembic, единственная начальная ревизия `backend/alembic/versions/e2df629b67e6_initial_schema.py`. Конфигурация — `alembic.ini`, окружение — `backend/alembic/env.py`.

Команды

```
make alembic-upgrade DATABASE_URL='postgresql+asyncpg://user:pass@host:5432/db'
make alembic-current DATABASE_URL='...'
make alembic-check        # согласованность моделей и миграций (tests/test_schema_sync.py)
```

Локально по умолчанию используется SQLite (`aiosqlite`) — так тесты и разработка не требуют внешней СУБД. Прод — PostgreSQL 16.

Правила

1. **Одна ревизия = одно намерение.** Сообщение отражает изменение схемы, а не номер задачи.
2. **Никогда не править применённую ревизию.** Нужна правка — новая ревизия.
3. **Модели и миграции обязаны совпадать.** `tests/test_schema_sync.py` сравнивает метаданные SQLAlchemy с историей; расхождение валит CI.
4. **Данные отделяются от схемы.** Демо-наполнение — `scripts/seed.py`, а не миграция.
5. **SQLite-only изменения недопустимы** как единственный вариант: прод — PostgreSQL.

Порядок обновления установленного экземпляра

1. Резервная копия (`pg_dump`; для файлов — бакет S3/MinIO).
2. `make alembic-upgrade` на новой версии образа.
3. `make alembic-current` — версия должна совпасть с head.
4. `make smoke-test` — `/healthz`, `/readyz`, фронтенд.
5. Проверить один сценарий вручную: доска, карточка, отчёт.

Автоматически это проверяется в `.github/workflows/portability.yml`: установка с нуля, повторный `upgrade head`, цикл backup → restore → smoke.